Layered Execution Trace

Algorithm Visualizations with Control Flow, Memory States, and Step-by-Step Transitions

A Comprehensive Guide to Classic Algorithms

April 2025

Table of Contents

Part 1: Weighted Interval Scheduling

1.1 Problem Overview

1.2 Greedy Approach

1.3 Naive Recursive Approach

1.4 Memoized DP Approach

1.5 Solution Reconstruction

1.6 Comparison Table

Part 2: Subset Sum Problem

2.1 Problem Overview

2.2 2D DP Approach

2.3 Space Optimized 1D DP

2.4 Backtracking

2.5 Comparison Table

Part 3: 0-1 Knapsack Problem

3.1 Problem Overview

3.2 Naive Recursive

3.3 Memoized DP

3.4 Iterative DP

3.5 Solution Reconstruction

Part 4: Graph Algorithms

4.1 Problem Setup

4.2 Kruskal's MST

4.3 Prim's MST

4.4 Dijkstra's Shortest Path

4.5 Dijkstra with Negative Weights

4.6 DSU Path Compression

Part 5: Greedy Interval Scheduling

5.1 Problem Overview

5.2 Sorting Phase

5.3 Greedy Selection

5.4 Exchange Argument Proof

Part 6: Fractional Knapsack

6.1 Problem Overview

6.2 Strategy 1: Minimum Weight

6.3 Strategy 2: Maximum Value

6.4 Strategy 3: Maximum Ratio

6.5 Proof of Optimality

Part 1: Weighted Interval Scheduling

1.1 Problem Overview

The **Weighted Interval Scheduling** problem asks us to select a subset of non-overlapping jobs to maximize the total weight (not just the count). Unlike the unweighted version where a simple greedy approach suffices, the weighted variant requires dynamic programming.

Problem Setup

Jobs are sorted by finish time. Each job has a start time, finish time, and weight.

Job	Interval	Weight
1	[1, 3]	5
2	[2, 5]	1
3	[4, 6]	8
4	[6, 7]	2

Optimal Solution: Job 1 + Job 3 + Job 4 = 5 + 8 + 2 = **15**

Greedy by weight (counterexample): Selecting only Job 3 = 8, which is not optimal.

1.2 Greedy Approach (Earliest Finish Time)

The greedy approach sorts by finish time and selects compatible jobs. While it happens to produce the optimal result for this specific example, greedy fails in general for weighted intervals.

Execution: greedyApproach()

Initial State:

```
jobs[] (sorted by finish):
idx | start | finish | weight
----|-------|--------|--------
  1 |   1   |    3   |   5
  2 |   2   |    5   |   1
  3 |   4   |    6   |   8
  4 |   6   |    7   |   2

selected = [1]
totalWeight = 5
last = 1
```

Iteration i = 2: jobs[2].start = 2 >= jobs[1].finish = 3? **No** (2 < 3). Skip (overlap).

Iteration i = 3: jobs[3].start = 4 >= jobs[1].finish = 3? **Yes**. Select Job 3. selected = [1, 3], total = 13, last = 3.

Iteration i = 4: jobs[4].start = 6 >= jobs[3].finish = 6? **Yes** (touching is OK). Select Job 4. selected = [1, 3, 4], total = 15, last = 4.

Greedy Approach State Transition

Step	i	Condition	Action	totalWeight
init	-	-	take jobs[1]	5
1	2	2 >= 3? No	skip	5
2	3	4 >= 3? Yes	take jobs[3]	13
3	4	6 >= 6? Yes	take jobs[4]	15
final	-	loop done	return	15

Critical Observation

Greedy produces 15 here *by coincidence*. It fails on: [1,2] w=5, [2,3] w=10, [3,4] w=6 where greedy picks [1,3]=11 but optimal is [2]=10... actually optimal = 10+6=16 with [2,3]. Greedy by earliest finish would pick [1,3] = 11.

1.3 Naive Recursive Approach (Exponential)

The recursive approach computes Opt(j) as the maximum of either including job j (and adding Opt(p[j])) or excluding job j (Opt(j-1)), where p[j] is the last compatible job before j.

Execution: recursiveComputeOpt(4)

```
Opt(4)                                      Call Stack (top to bottom):
 /                    \                    recursive(4) computing...
include job 4        exclude job 4         recursive(3) (waiting)
weight=2+Opt(2)      Opt(3)                recursive(2) (waiting)
                     /            \        recursive(1) (waiting)
               include 3       exclude 3   recursive(0) BASE CASE
               weight=8+       Opt(2)
               Opt(1)
              /        \
        include 1    exclude 1
        weight=5+    Opt(0)=0
        Opt(0)=5

Base: Opt(0) = 0
```

Recursion Tree Evaluation:

```
Opt(1) = max(5 + Opt(0), Opt(0)) = max(5, 0) = 5
Opt(2) = max(1 + Opt(0), Opt(1)) = max(1, 5) = 5
Opt(3) = max(8 + Opt(1), Opt(2)) = max(13, 5) = 13
Opt(4) = max(2 + Opt(2), Opt(3)) = max(7, 13) = 15 (Wait - correction below)
```

Critical Flaw

Same subproblems are recomputed exponentially. Opt(2) is computed 2 times, Opt(1) is computed 3 times. Time complexity: **$O(2^n)$**. Skipped in practice for n > 35.

1.4 Memoized DP Approach (Efficient)

Memoization stores computed results in a cache M[], turning exponential time into linear.

Execution: memoizedComputeOpt(4) with M[] cache

Initial State: M = [0, -1, -1, -1, -1] (M[0] = 0 as base case)

p[] (compatible indices): p = [0, 0, 0, 1, 2]

Computing M[4]:

```
incl = jobs[4].weight + M[p[4]] = 2 + M[2] = 2 + 5 = 7
excl = M[3] = 13
M[4] = max(7, 13) = 15 (with corrected p[4])
```

Corrected Computation with p[4]=3:

```
j=1: incl = 5 + M[0] = 5, excl = M[0] = 0  =>  M[1] = 5
j=2: incl = 1 + M[0] = 1, excl = M[1] = 5  =>  M[2] = 5
j=3: incl = 8 + M[1] = 13, excl = M[2] = 5 =>  M[3] = 13
j=4: incl = 2 + M[3] = 15, excl = M[2] = 5 =>  M[4] = 15
```

Memoized DP Computation Trace

j	M[j]	incl	excl	max	Notes
0	0	-	-	0	base
1	5	5 + M[0] = 5	M[0] = 0	5	take j1
2	5	1 + M[0] = 1	M[1] = 5	5	skip j2
3	13	8 + M[1] = 13	M[2] = 5	13	take j3
4	15	2 + M[3] = 15	M[2] = 5	15	take j4

Key Insight

Each M[j] is computed **exactly once**, giving **O(n)** total time. Memoization transforms exponential complexity into linear.

1.5 Solution Reconstruction (Backtracking)

After computing M[], we backtrack to determine which jobs were selected.

Execution: reconstructSolution(4)

Using: M = [0, 5, 5, 13, 15]

j = 4: jobs[4].weight + M[p[4]] = 2 + M[3] = 15 >= M[3] = 13? **Yes**. Include Job 4. Go to p[4] = 3.

j = 3: jobs[3].weight + M[p[3]] = 8 + M[1] = 13 >= M[2] = 5? **Yes**. Include Job 3. Go to p[3] = 1.

j = 1: jobs[1].weight + M[p[1]] = 5 + M[0] = 5 >= M[0] = 0? **Yes**. Include Job 1. Go to p[1] = 0.

j = 0: Base case. Return.

Reconstruction Trace

j	Condition	Decision	Solution
4	15 >= 13?	Yes, include	[4]
3	13 >= 5?	Yes, include	[4, 3]
1	5 >= 0?	Yes, include	[4, 3, 1]
0	base case	return	[4, 3, 1] reversed: [1, 3, 4]

Final Solution: Jobs [1, 3, 4] with total weight **15**.

1.6 Comparison Table

Weighted Interval Scheduling Approaches

Approach	Time	Space	Key Characteristic
Greedy	O(n log n) sorting	O(n)	Fast but not optimal for weighted case
Naive Recursive	$O(2^n)$	O(n) stack	Exponential, explores all subsets
Memoized DP	O(n log n)	O(n)	Optimal, each subproblem solved once

Part 2: Subset Sum Problem

2.1 Problem Overview

The **Subset Sum** problem asks: given a set of integers and a target value, can we select a subset that sums to exactly the target?

> **Input**
>
> **Array:** arr = [2, 3, 7, 8, 10]
>
> **Target:** 11
>
> **Answer:** Yes, {3, 8} = 11

2.2 2D DP Approach (Full Table)

Let dp[i][j] = true if a subset of the first i elements can sum to j.

Execution: subsetSum() with DP Table

Recurrence:

```
dp[i][j] = dp[i-1][j] || (j >= arr[i-1] && dp[i-1][j - arr[i-1]])
         (exclude)         (include, if arr[i-1] fits)
```

Base Cases:

```
dp[0][0] = true       (empty set sums to 0)
dp[0][j] = false for j > 0  (no elements, no positive sum)
dp[i][0] = true for all i   (empty subset always sums to 0)
```

Processing element 2 (arr[0]):

```
i=1: dp[1][2] = dp[0][2] || dp[0][0] = false || true = true
```

Processing element 3 (arr[1]):

```
i=2: dp[2][3] = dp[1][3] || dp[1][0] = false || true = true
      dp[2][5] = dp[1][5] || dp[1][2] = false || true = true  (2+3=5)
```

Processing element 7 (arr[2]):

```
i=3: dp[3][7] = dp[2][7] || dp[2][0] = false || true = true
      dp[3][9] = dp[2][9] || dp[2][2] = false || true = true  (2+7=9)
      dp[3][10] = dp[2][10] || dp[2][3] = false || true = true (3+7=10)
```

Processing element 8 (arr[3]) - Key Moment:

```
i=4: dp[4][11] = dp[3][11] || dp[3][3] = false || true = true
      (3 + 8 = 11, FOUND!)
```

Processing element 10 (arr[4]):

```
i=5: dp[5][11] = dp[4][11] || dp[4][1] = true || false = true
```

Final DP Table (dp[i][j])

i\j	0	1	2	3	4	5	6	7	8	9	10	11
0	T	F	F	F	F	F	F	F	F	F	F	F
1 (2)	T	F	T	F	F	F	F	F	F	F	F	F
2 (3)	T	F	T	T	F	T	F	F	F	F	F	F
3 (7)	T	F	T	T	F	T	F	T	F	T	T	F
4 (8)	T	F	T	T	F	T	F	T	T	T	T	T
5 (10)	T	F	T	T	F	T	F	T	T	T	T	T

State Transition Steps

Step	i	j	Action
init	-	-	dp[*][0] = true
1	1	2	Take element 2
2	2	3	Take element 3
3	2	5	2 + 3 = 5
4	3	7	Take element 7
5	3	9	2 + 7 = 9
6	3	10	3 + 7 = 10
7	4	8	Take element 8
8	4	11	3 + 8 = 11 (FOUND)

2.3 Space Optimized 1D DP

Since dp[i][j] depends only on the previous row, we can optimize to a single 1D array.

Execution: subsetSumOptimized()

Critical: Traverse **backwards** to prevent using updated values from the same iteration.

```
Initial: dp = [T, F, F, F, F, F, F, F, F, F, F, F]

i=0, arr[0]=2:  (traverse j from 11 down to 2)
  j=2: dp[2] = dp[2] || dp[0] = F || T = T
  dp = [T, F, T, F, F, F, F, F, F, F, F, F]

i=1, arr[1]=3:
  j=5: dp[5] = dp[5] || dp[2] = F || T = T  (2+3=5)
  j=3: dp[3] = dp[3] || dp[0] = F || T = T
  dp = [T, F, T, T, F, T, F, F, F, F, F, F]

i=2, arr[2]=7:
  j=10: dp[10] = dp[10] || dp[3] = F || T = T  (3+7=10)
  j=9:  dp[9]  = dp[9]  || dp[2] = F || T = T  (2+7=9)
  j=7:  dp[7]  = dp[7]  || dp[0] = F || T = T
  dp = [T, F, T, T, F, T, F, T, F, T, T, F]

i=3, arr[3]=8:
  j=11: dp[11] = dp[11] || dp[3] = F || T = T  (3+8=11 FOUND!)
  dp = [T, F, T, T, F, T, F, T, F, T, T, T]
```

Why Backward Traversal?

Forward pass would **overwrite** dp[j] and then use the *new* value when computing dp[j + arr[i]]. Backward traversal ensures each element is used at most once per iteration, preserving the 0-1 constraint.

2.4 Backtracking (Finding the Actual Subset)

Starting from dp[n][target], compare dp[i][j] with dp[i-1][j] to determine if element i was used.

Execution: subsetSumWithPath()

```
i=5, j=11: dp[5][11] = T, dp[4][11] = T
           Equal => did NOT take arr[4]=10. i -> 4

i=4, j=11: dp[4][11] = T, dp[3][11] = F
           NOT equal => TOOK arr[3]=8! Subset: [8], j = 11-8 = 3

i=3, j=3:  dp[3][3] = T, dp[2][3] = T
           Equal => did NOT take arr[2]=7. i -> 2

i=2, j=3:  dp[2][3] = T, dp[1][3] = F
           NOT equal => TOOK arr[1]=3! Subset: [8, 3], j = 3-3 = 0

j=0 => STOP! Reverse: [3, 8]
```

Backtracking Trace

i	j	dp[i][j] vs dp[i-1][j]	Action	Subset
5	11	T == T	Skip 10	[]
4	11	T != F	Take 8	[8]
3	3	T == T	Skip 7	[8]
2	3	T != F	Take 3	[8, 3]
1	0	Stop	Return	[8, 3] -> [3, 8]

Final Output: Subset = {3, 8}, Sum = 11.

2.5 Comparison Table

Subset Sum Approaches

Approach	Time	Space	When to Use
2D DP Table	O(n x target)	O(n x target)	Need to reconstruct the subset
1D Optimized	O(n x target)	O(target)	Only need yes/no answer
Backtracking	O(n + target)	O(n x target)	Requires 2D table to trace

Key Insights

1. dp[i][j] depends ONLY on dp[i-1][*] (previous row), enabling 1D optimization.
2. Backtracking works by comparing dp[i][j] with dp[i-1][j]: if equal, element i was NOT used; if different, it WAS used.
3. Backward traversal in 1D version prevents "reuse" of the same element within one iteration.

Part 3: 0-1 Knapsack Problem

3.1 Problem Setup

In the **0-1 Knapsack** problem, each item can be taken at most once. Given a capacity constraint, maximize total value.

Input

ID	Value	Weight
1	3	2
2	4	3
3	5	4

Capacity: 5 **Optimal:** {1, 2} = 7 value (weight 5)

3.2 Naive Recursive (Exponential Explosion)

Execution: knapsackRecursive(3, 5)

```
knap(3, 5): val=5, w=4 <= 5? Yes -> branch
  include: 5 + knap(2, 1)
    item 2, w=3 > 1? Yes -> skip
      knap(1, 1): item 1, w=2 > 1? Yes -> skip
        knap(0, 1) = 0 (base)
      return 0
    return 0
  exclude: knap(2, 5)
    item 2, w=3 <= 5? Yes -> branch
      include: 4 + knap(1, 2)
        item 1, w=2 <= 2? Yes -> branch
          include: 3 + knap(0, 0) = 3
          exclude: knap(0, 2) = 0
        return max(3, 0) = 3
      exclude: knap(1, 5)
        ...

CRITICAL: knap(1, 2) is computed TWICE
Total calls: exponential in n
```

Recursive Call Trace

Call	n	Capacity	Decision	Return
1	3	5	branch	max(incl, excl)
2	2	1	w=3 > 1, skip	knap(1, 1) = 0
3	1	1	w=2 > 1, skip	knap(0, 1) = 0
4	2	5	w=3 <= 5	max(4+knap(1,2), knap(1,5))
5	1	2	w=2 <= 2	max(3+knap(0,0), knap(0,2)) = 3
6	0	0	base case	0
7	1	5	w=2 <= 5	max(3+knap(0,3), knap(0,5)) = 3

Complexity

$O(2^n)$: each item has two choices (take or not take). The naive recursive approach recomputes the same subproblems exponentially.

3.3 Memoized DP (Top-Down with Cache)

A memo table stores already-computed states to avoid recomputation.

Execution: knapsackMemoized(3, 5)

Initial: memo[4][6] all -1 (uncomputed). memo[0][*] = 0.

```
memo[3][5] = -1 -> compute:
  w=4 <= 5 -> branch
  incl: 5 + memo[2][1]
    memo[2][1] = -1 -> compute:
      w=3 > 1 -> skip -> memo[1][1]
        memo[1][1] = -1 -> compute:
          w=2 > 1 -> skip -> memo[0][1] = 0
          Cache: memo[1][1] = 0
        Cache: memo[2][1] = 0

  excl: memo[2][5]
    memo[2][5] = -1 -> compute:
      w=3 <= 5 -> branch
      Need memo[1][2] and memo[1][5]...
        memo[1][2] = max(3+memo[0][0], memo[0][2]) = max(3, 0) = 3
        memo[1][5] = max(3+memo[0][3], memo[0][5]) = max(3, 0) = 3
      memo[2][5] = max(4+3, 3) = 7

  memo[3][5] = max(5+0, 7) = 7
```

Final memo Table

n\w	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7

Cache Hit Example

If called again: knapsackMemoized(1, 5) returns memo[1][5] = 3 **instantly** without recomputation.

3.4 Iterative DP (Bottom-Up Table Filling)

Execution: knapsackIterative(3, 5)

Recurrence:

```
if weight[i] > w:  dp[i][w] = dp[i-1][w]
else:                    dp[i][w] = max(dp[i-1][w], value[i] + dp[i-1][w-weight[i]])
                       (skip i)          (take i, use remaining capacity)
```

After i=1 (item 1: val=3, w=2):

```
dp[1][2] = max(dp[0][2], 3+dp[0][0]) = max(0, 3) = 3
dp[1][3] = max(dp[0][3], 3+dp[0][1]) = 3
dp[1][4] = 3, dp[1][5] = 3
```

After i=2 (item 2: val=4, w=3):

```
dp[2][3] = max(dp[1][3]=3, 4+dp[1][0]=4) = 4
dp[2][5] = max(dp[1][5]=3, 4+dp[1][2]=7) = 7
```

After i=3 (item 3: val=5, w=4):

```
dp[3][4] = max(dp[2][4]=4, 5+dp[2][0]=5) = 5
dp[3][5] = max(dp[2][5]=7, 5+dp[2][1]=5) = 7
```

Final DP Table

	w=0	w=1	w=2	w=3	w=4	w=5
i=0 (no items)	0	0	0	0	0	0
i=1 (item 1)	0	0	3	3	3	3
i=2 (items 1-2)	0	0	3	4	4	7
i=3 (items 1-3)	0	0	3	4	5	7

Answer: dp[3][5] = **7**

3.5 Solution Reconstruction (Backtracking)

Execution: reconstructSolutionIterative(3, 5, dp)

```
i=3, w=5: dp[3][5]=7, dp[2][5]=7
          Equal -> Item 3 was NOT taken! i -> 2

i=2, w=5: dp[2][5]=7, dp[1][5]=3
          NOT equal -> Item 2 WAS taken!
          selected += [2], w = 5 - 3 = 2

i=1, w=2: dp[1][2]=3, dp[0][2]=0
          NOT equal -> Item 1 WAS taken!
          selected += [1], w = 2 - 2 = 0

i=0 or w=0 -> STOP!
Reverse selected: [2, 1] -> [1, 2]
```

Backtracking Trace

Step	i	w	Decision
1	3	5	dp[3][5]==dp[2][5]? Skip item 3
2	2	5	dp[2][5]!=dp[1][5]? TAKE item 2
3	1	2	dp[1][2]!=dp[0][2]? TAKE item 1
4	0	0	STOP
final	-	-	Reverse -> [1, 2]

Result: Items [1, 2], Value = 3 + 4 = 7, Weight = 2 + 3 = 5 <= 5.

3.6 Comparison of All Approaches

0-1 Knapsack Approaches

Approach	Time	Space	Characteristics
Naive Recursive	$O(2^n)$	O(n) stack	Exponential, elegant but impractical
Memoized DP (Top-Down)	O(n x W)	O(n x W) table + stack	Fills only needed states
Iterative DP (Bottom-Up)	O(n x W)	O(n x W) table	Fills entire table, no recursion overhead
Space Optimized	O(n x W)	O(W)	1D array, loses reconstruction path

Key Recurrence

```
dp[i][w] = dp[i-1][w]                          if w_i > w (can't fit)
dp[i][w] = max(dp[i-1][w], v_i + dp[i-1][w-w_i]) if w_i <= w
                (skip i)      (take i)
```

This is the "0-1" constraint: each item used 0 times OR 1 time. Unlike unbounded knapsack where items can be reused.

Part 4: Graph Algorithms (MST and Shortest Path)

4.1 Problem Setup

Consider an undirected weighted graph with 6 nodes (A-F) and 9 edges.

Edge List (sorted by weight for Kruskal)

Edge	Weight	Nodes
1	1	C - D
2	2	A - C
3	2	D - F
4	3	B - E
5	4	A - B
6	5	D - E
7	6	E - F
8	7	A - D

4.2 Kruskal's MST (Union-Find)

Kruskal's algorithm greedily selects edges in order of increasing weight, adding an edge if it does not form a cycle.

Execution: kruskalMST()

Initial DSU: Parent = [A, A, A, A, A, A], Rank = [0, 0, 0, 0, 0, 0]. Each node is its own component.

Edge 1: C-D (weight 1)

```
find(C) = C, find(D) = D -> Different roots! -> unite(C, D)
parent[D] = C, rank[C] = 1
Components: {A}, {B}, {C,D}, {E}, {F}
Cost: 1, Edges: 1
```

Edge 2: A-C (weight 2)

```
find(A) = A, find(C) = C -> Different roots! -> unite(A, C)
parent[C] = A, rank[A] = 1
Components: {A,C,D}, {B}, {E}, {F}
Cost: 3, Edges: 2
```

Edge 3: D-F (weight 2)

```
find(D) = find(C) = find(A) = A, find(F) = F -> unite(A, F)
parent[F] = A
Components: {A,C,D,F}, {B}, {E}
Cost: 5, Edges: 3
```

Edge 4: B-E (weight 3)

```
find(B) = B, find(E) = E -> unite(B, E)
parent[E] = B
Components: {A,C,D,F}, {B,E}
Cost: 8, Edges: 4
```

Edge 5: A-B (weight 4)

```
find(A) = A, find(B) = B -> unite(A, B)
parent[B] = A
Components: {A,B,C,D,E,F} -> ALL CONNECTED!
Cost: 12, Edges: 5 = V-1
```

STOP: edgesIncluded == V-1 (5 edges). Remaining edges (D-E, E-F, A-D) are skipped as they would form cycles.

Kruskal State Transition

Step	Edge	Weight	Action	Total Cost
1	C-D	1	Take (union)	1
2	A-C	2	Take (union)	3
3	D-F	2	Take (union)	5
4	B-E	3	Take (union)	8
5	A-B	4	Take (union)	12
6	D-E	5	Skip (cycle)	12
7	E-F	6	Skip (cycle)	12
8	A-D	7	Skip (cycle)	12

Final MST: Total Cost = 1 + 2 + 2 + 3 + 4 = **12**

4.3 Prim's MST (Priority Queue)

Prim's algorithm grows the MST from a starting node, always extending with the cheapest edge to a fringe node.

Execution: primMST("A")

Initial: key = [0, inf, inf, inf, inf, inf], parent = [-1, -1, -1, -1, -1, -1], inMST = [F, F, F, F, F, F]. PQ = {(0, A)}.

Step 1: Extract A (key=0)

```
inMST[A] = true, Total = 0
Neighbors: B(key=4), C(key=2), D(key=7)
PQ: {(2, C), (4, B), (7, D)}
```

Step 2: Extract C (key=2)

```
inMST[C] = true, Edge A-C (weight 2), Total = 2
Neighbor D: key[D]=7 > 1? -> key[D]=1, parent[D]=C
PQ: {(1, D), (4, B)}
```

Step 3: Extract D (key=1)

```
inMST[D] = true, Edge C-D (weight 1), Total = 3
Neighbors: F(key=2), E(key=5)
PQ: {(2, F), (4, B), (5, E)}
```

Step 4: Extract F (key=2)

```
inMST[F] = true, Edge D-F (weight 2), Total = 5
Neighbor E: key=6 < 5? No, skip
PQ: {(4, B), (5, E)}
```

Step 5: Extract B (key=4)

```
inMST[B] = true, Edge A-B (weight 4), Total = 9
Neighbor E: key=3 < 5? -> key[E]=3, parent[E]=B
PQ: {(3, E)}
```

Step 6: Extract E (key=3)

```
inMST[E] = true, Edge B-E (weight 3), Total = 12
All 6 nodes in MST. STOP!
```

Total: 2 + 1 + 2 + 4 + 3 = **12** (same as Kruskal).

4.4 Dijkstra's Shortest Path

Dijkstra's algorithm finds the shortest path from a source node to all other nodes.

Execution: dijkstra("A")

Initial: dist = [0, inf, inf, inf, inf, inf], parent = [-1, -1, -1, -1, -1, -1], inS = [F, F, F, F, F, F]. PQ = {(0, A)}.

Step 1: Settle A (dist=0)

```
Relax: B(0+4=4), C(0+2=2), D(0+7=7)
PQ: {(2, C), (4, B), (7, D)}
```

Step 2: Settle C (dist=2)

```
Path: A->C. Relax D: dist[C]+1=3 < 7? -> dist[D]=3, parent[D]=C
PQ: {(3, D), (4, B)}
```

Step 3: Settle D (dist=3)

```
Path: A->C->D. Relax F(5), E(8)
PQ: {(4, B), (5, F), (8, E)}
```

Step 4: Settle B (dist=4)

```
Path: A->B. Relax E: 0+4+3=7 < 8? -> dist[E]=7, parent[E]=B
PQ: {(5, F), (7, E)}
```

Step 5: Settle F (dist=5)

```
Path: A->C->D->F. Relax E: 5+6=11 > 7, skip
```

Step 6: Settle E (dist=7)

```
Path: A->B->E. All nodes settled!
```

Final Shortest Paths from A

Node	Distance	Path
B	4	A -> B
C	2	A -> C
D	3	A -> C -> D
E	7	A -> B -> E
F	5	A -> C -> D -> F

Prim vs Dijkstra at Node D

Prim (MST): key[D] = 1 (cheapest edge connecting D to the growing tree). Edge C-D chosen as the minimum weight edge connecting {A,C} to {D,F,B,E}.

Dijkstra (SP): dist[D] = 3 (total cost from A to D). Path A->C->D = 2+1 = 3.

Key Difference: Prim minimizes edge weight to connect components; Dijkstra minimizes total path cost from the source.

4.5 Dijkstra Fails with Negative Weights

Dijkstra's greedy settlement assumes that once a node's shortest distance is finalized, it cannot be improved. This fails with negative edges.

Counterexample Graph (3 nodes)

Edges: A->B (5), A->C (2), B->C (-10)

Actual Shortest: A->B->C = 5 + (-10) = -5 (cheaper than direct A->C = 2)

Execution: dijkstra("A") with Negative Edge

Step 1: Settle A

```
Relax: dist[B]=5, dist[C]=2. PQ: {(2, C), (5, B)}
```

Step 2: Settle C (dist=2)

```
inS[C]=T, dist[C]=2 (LOCKED!)
C has no outgoing edges. PQ: {(5, B)}
```

Step 3: Settle B (dist=5)

```
inS[B]=T, dist[B]=5 (LOCKED!)
Relax B->C: dist[C] = min(2, 5+(-10)) = min(2, -5) = -5
BUT inS[C]=T -> SKIP! Cannot update settled node!
```

Result: dist[C] = 2, but the actual shortest is **-5** via A->B->C.

Why Dijkstra Fails

Dijkstra's greedy invariant "once settled, distance is final" holds ONLY for non-negative weights. With negative edges, a later-discovered path through a heavier initial edge may yield a better total. The fix is **Bellman-Ford**, which relaxes all edges V-1 times without a settled set.

4.6 DSU Path Compression

Disjoint Set Union with path compression makes each find() operation nearly constant time.

Execution: find(7) with Path Compression

Setup: After unions, parent = [0, 0, 0, 2, 0, 4, 4, 6] (node 0 is root of everything).

Initial tree height before compression: find(7) requires traversing 7->6->4->0 (3 hops).

Recursive Descent:

```
find(7): parent[7]=6 != 7 -> recurse find(6)
find(6): parent[6]=4 != 6 -> recurse find(4)
find(4): parent[4]=0 != 4 -> recurse find(0)
find(0): parent[0]=0 -> return 0 (root found!)
```

Unwinding + Compression:

```
find(0) returns 0 to find(4): parent[4]=0 (unchanged, already root child)
find(4) returns 0 to find(6): parent[6]=0 <- COMPRESSED!
find(6) returns 0 to find(7): parent[7]=0 <- COMPRESSED!
find(7) returns 0
```

After find(7): parent = [0, 0, 0, 2, 0, 4, 0, 0]

Nodes 7 and 6 now point directly to the root! Future finds on these nodes are O(1).

Path Compression Trace

Call	Recursive Stack	Path Taken	Side Effect
find(7)	[7]	7->6	none yet
find(6)	[7, 6]	6->4	none yet
find(4)	[7, 6, 4]	4->0	none yet
find(0)	[7, 6, 4, 0]	0 (root)	Returns 0
unwind	[7, 6, 4]	4->0	parent[4]=0 (unchanged)
unwind	[7, 6]	6->0	parent[6]=0 COMPRESSED
unwind	[7]	7->0	parent[7]=0 COMPRESSED

Amortized Complexity

Without compression: find = O(tree height) = O(n) worst case.
With compression: find = **O(alpha(n))**, where alpha is the inverse Ackermann function. It grows incredibly slowly (alpha(10^600) is about 4), so for all practical purposes it is less than 5. Essentially constant time!

4.7 Algorithm Comparison

Graph Algorithm Comparison

Algorithm	Strategy	Data Structure	Key Difference
Kruskal's MST	Edge-based greedy	Sort + DSU	Sort all edges, add if no cycle
Prim's MST	Node-based greedy	Priority Queue	Grow from start, pick cheapest edge to fringe
Dijkstra's SP	Node-based greedy	Priority Queue	Same structure but minimizes path cost, not edge cost

Time Complexity

Algorithm	With Optimization	Without Optimization
Kruskal's MST	O(E log E)	O(E log E) sort + alpha(V) union
Prim's MST	O(E log V)	O(V^2) with array
Dijkstra's SP	O(E log V)	O(V^2) with array

Part 5: Greedy Interval Scheduling

5.1 Problem Overview

The **Greedy Interval Scheduling** problem selects the maximum number of non-overlapping jobs. Unlike the weighted version, we only care about count, not value.

Input: 6 Jobs (unsorted)

ID	Start	Finish	Interval
1	1	3	[1, 3]
2	2	4	[2, 4]
3	3	5	[3, 5]
4	0	6	[0, 6]
5	5	7	[5, 7]
6	8	9	[8, 9]

Goal: Select maximum number of compatible (non-overlapping) jobs.

Key Insight: Sort by finish time, then greedily pick compatible jobs.

5.2 Sorting Phase

Execution: sort(jobs.begin()+1, jobs.end(), compareJobs)

Sorting by finish time in ascending order:

Index	ID	Start	Finish
1	1	1	3
2	2	2	4
3	3	3	5
4	4	0	6
5	5	5	7
6	6	8	9

5.3 Greedy Selection Phase

Execution: greedyIntervalScheduling()

Initial: count = 1, lastFinish = 3 (jobs[1].finish), selected = [1]

i = 2: jobs[2] = [2, 4] (id=2). start=2 >= lastFinish=3? **No** (2 < 3). Skip (overlaps with job 1).

```
Visual: [1====3]  [2======4] <- overlaps with job 1
```

i = 3: jobs[3] = [3, 5] (id=3). start=3 >= lastFinish=3? **Yes** (touching is OK). Select.

```
count = 2, lastFinish = 5, selected = [1, 3]
Visual: [1====3]  [3======5] <- touching is compatible!
```

i = 4: jobs[4] = [0, 6] (id=4). start=0 >= lastFinish=5? **No**. Skip (major overlap).

```
Visual: [0==========6] <- spans everything!
```

i = 5: jobs[5] = [5, 7] (id=5). start=5 >= lastFinish=5? **Yes**. Select.

```
count = 3, lastFinish = 7, selected = [1, 3, 5]
Visual: [1====3]  [3====5]  [5======7]
```

i = 6: jobs[6] = [8, 9] (id=6). start=8 >= lastFinish=7? **Yes**. Select.

```
count = 4, lastFinish = 9, selected = [1, 3, 5, 6]
Visual: [1====3]  [3====5]  [5====7]  [8==9]
```

Result: 4 jobs selected: [1, 3, 5, 6] with intervals [1,3], [3,5], [5,7], [8,9].

Greedy Selection State Transition

Step	i	Job [s, f]	Start vs Last	Condition	Action	Count
init	-	-	-	-	Select jobs[1]	1
1	2	[2, 4] id2	2 >= 3	No	Skip	1
2	3	[3, 5] id3	3 >= 3	Yes	Take	2
3	4	[0, 6] id4	0 >= 5	No	Skip (long job)	2
4	5	[5, 7] id5	5 >= 5	Yes	Take	3
5	6	[8, 9] id6	8 >= 7	Yes	Take	4
final	-	-	loop end	-	Return {4, [1,3,5,6]}	4

5.4 Why Greedy Works (Exchange Argument)

Proof: Greedy Choice is Always Safe

Claim: There exists an optimal solution that includes the job with the **earliest finish time**.

Suppose an optimal solution picks some job J first, while the greedy choice G has the earliest finish time. Since G finishes before or at the same time as J (G was earliest), we can replace J with G without creating conflicts. The solution size stays the same.

Inductive Step: After picking G, remove all jobs overlapping G. Solve the remaining subproblem with the same greedy strategy. By induction, greedy is optimal for the subproblem. Combine: G + optimal(subproblem) = optimal(original).

Comparison: What If We Sorted Differently?

By start time (descending): Pick [8,9], then all others overlap. Result: 1 job only. Far from optimal!

By longest duration first: Job 4 [0,6] is longest, but blocks everything else. Result: 1 job.

Conclusion: Sorting by **earliest finish time** is the key. It leaves maximum remaining time for other jobs.

Comparison: This Problem vs Weighted Version

Aspect	This Problem (max count)	Weighted Version (max weight)
Greedy	Works!	Fails!
Complexity	O(n log n)	Needs DP: O(n^2) or O(n log n) with BS

Complexity Analysis

Phase	Time	Space
Input Reading	O(n)	O(n) for jobs array
Sorting	O(n log n)	O(1) extra (in-place)
Greedy Scan	O(n)	O(k) for selected (k = result)
Total	**O(n log n)**	**O(n)**

Part 6: Fractional Knapsack

6.1 Problem Setup

In the **Fractional Knapsack** problem, we can take fractions of items. Unlike 0-1 knapsack, greedy by value/weight ratio is optimal here.

Input: Capacity = 50, 4 Items

ID	Value	Weight	Ratio (v/w)
1	60	10	6.00
2	100	20	5.00
3	120	30	4.00
4	15	5	3.00

Key Difference: We can take any fraction of an item and receive proportional value.

6.2 Strategy 1: Minimum Weight First

Execution: runKnapsackStrategy(items, 50, "Minimum Weight", cmpMinWeight)

Sorted by weight ascending: 4(5w), 1(10w), 2(20w), 3(30w)

```
curW=0, profit=0

Item 4 (w=5):  curW+5=5 <= 50 -> TAKE ALL
  curW=5, profit=15

Item 1 (w=10): curW+10=15 <= 50 -> TAKE ALL
  curW=15, profit=75

Item 2 (w=20): curW+20=35 <= 50 -> TAKE ALL
  curW=35, profit=175

Item 3 (w=30): curW+30=65 > 50 -> TAKE FRACTION
  remaining = 50-35 = 15
  fraction = 15/30 = 0.5 (50%)
  value = 120 * 0.5 = 60
  curW=50, profit=235
```

Result: Profit = **235.00**, Items: 4(full), 1(full), 2(full), 3(50%)

6.3 Strategy 2: Maximum Value First

Execution: runKnapsackStrategy(items, 50, "Maximum Value", cmpMaxValue)

Sorted by value descending: 3(120v), 2(100v), 1(60v), 4(15v)

```
curW=0, profit=0

Item 3 (w=30): curW+30=30 <= 50 -> TAKE ALL
  curW=30, profit=120

Item 2 (w=20): curW+20=50 <= 50 -> TAKE ALL
  curW=50, profit=220

Item 1 (w=10): curW+10=60 > 50 -> No capacity! SKIP

Item 4 (w=5): curW+5=55 > 50 -> No capacity! SKIP
```

Result: Profit = **220.00**, Items: 3(full), 2(full)

Why it fails: Item 3 has high total value but poor "density" (ratio=4). It consumes 60% of capacity for only 120 value. Items 1+2 give 160 value in only 30 weight (better density).

6.4 Strategy 3: Maximum Ratio (Value/Weight) - Optimal

Execution: runKnapsackStrategy(items, 50, "Max Unit Value", cmpMaxRatio)

Sorted by ratio descending: 1(r=6), 2(r=5), 3(r=4), 4(r=3)

```
curW=0, profit=0

Item 1 (r=6.0, w=10, v=60): curW+10=10 <= 50 -> TAKE ALL
  curW=10, profit=60

Item 2 (r=5.0, w=20, v=100): curW+20=30 <= 50 -> TAKE ALL
  curW=30, profit=160

Item 3 (r=4.0, w=30, v=120): curW+30=60 > 50 -> TAKE FRACTION
  remaining = 50-30 = 20
  fraction = 20/30 = 66.67%
  value = 120 * 0.6667 = 80.00
  curW=50, profit=240

Item 4 (r=3.0): No capacity, skip
```

Result: Profit = **240.00** (BEST!), Items: 1(full), 2(full), 3(66.67%)

Strategy Comparison

Strategy	Profit	Selected Items
Minimum Weight First	235.00	4(full), 1(full), 2(full), 3(50%)
Maximum Value First	220.00	3(full), 2(full)
Maximum Ratio First	**240.00**	1(full), 2(full), 3(66.67%) OPTIMAL

6.5 Why Maximum Ratio is Optimal (Proof Sketch)

Exchange Argument for Fractional Knapsack

Claim: Sorting by value/weight ratio and greedily taking items in that order yields the optimal solution.

Suppose greedy picks items in ratio order: r1 >= r2 >= r3... Consider any optimal solution O that differs from greedy G. Find the first position where O and G differ. At this position, G has higher-ratio material than O. We can exchange some lower-ratio material in O with higher-ratio material from G (same weight). Since r_high > r_low, the value increases or stays the same. Thus O cannot be better than G.

Crucial Difference from 0-1 Knapsack

Fractional knapsack allows this exchange because items are divisible; we can always swap partial items to improve the ratio.

0-1 knapsack fails because items are indivisible. We cannot take 0.5 of item 3 to make room for item 1. Greedy by ratio might skip a slightly lower-ratio item that would free up perfect space for multiple better items.

Example where 0-1 greedy fails: Capacity 50, items: (1) v=60,w=10,r=6; (2) v=100,w=20,r=5; (3) v=120,w=30,r=4. Greedy by ratio picks 1+2 = 160 value (30 weight), can't fit 3. Optimal: pick 2+3 = 220 value (50 weight).

Control Flow Comparison

Step	Min Weight	Max Value	Max Ratio
Sort	by weight asc	by value desc	by ratio desc
1st	Take 4 (5w, 15v)	Take 3 (30w, 120v)	Take 1 (10w, 60v)
2nd	Take 1 (10w, 60v)	Take 2 (20w, 100v)	Take 2 (20w, 100v)
3rd	Take 2 (20w, 100v)	Try 1: 0 left	Take 3 67% (20w, 80v)
4th	Take 3 50% (15w, 60v)	Try 4: 0 left	Try 4: 0 left
Result	**235.00**	**220.00**	**240.00 OPTIMAL**

6.6 Key Insights

1. **Greedy works for fractional, fails for 0-1:** Fractional items can always be exchanged to improve the solution; 0-1 items may force suboptimal gaps.
2. **The ratio r = v/w is the "density":** It measures value per unit weight. Always pick the densest remaining item first.
3. **Complexity:** Sorting O(n log n) + Greedy scan O(n) = **O(n log n)** time, **O(1)** extra space.